AURORA ROCKETRY

# ATLAS

# FLIGHT SOFTWARE

## ARCHITECTURE & CODEBASE INTRODUCTION

Rookie onboarding · Incremental architecture walkthrough



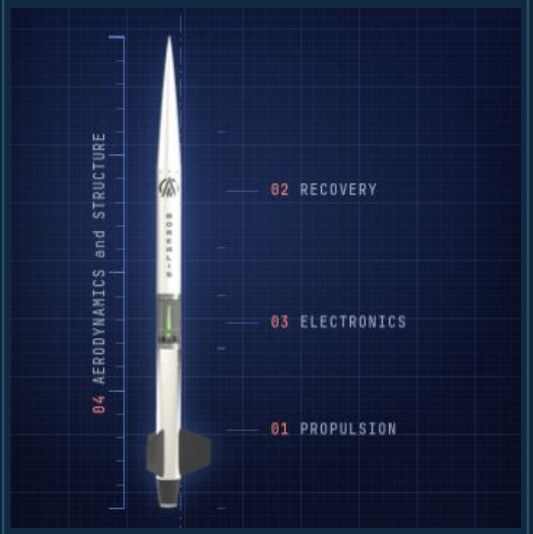
SOFTWARE DEPARTMENT

NOVEMBER 2026

# Meet the Rockets

Four names you will hear around Aurora Rocketry.

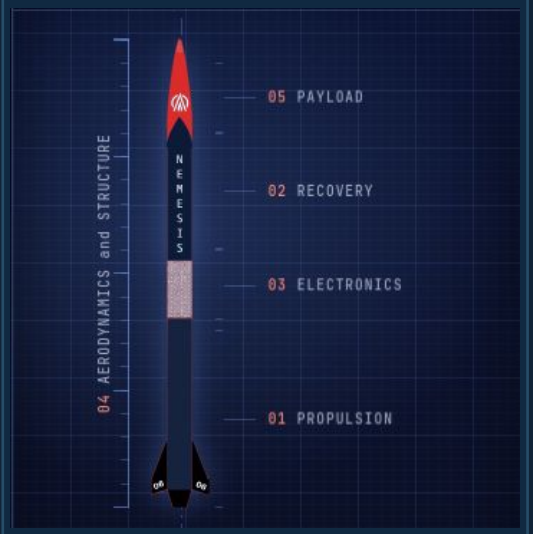
## BOREALIS




**First-generation rocket**

Initial systems validation

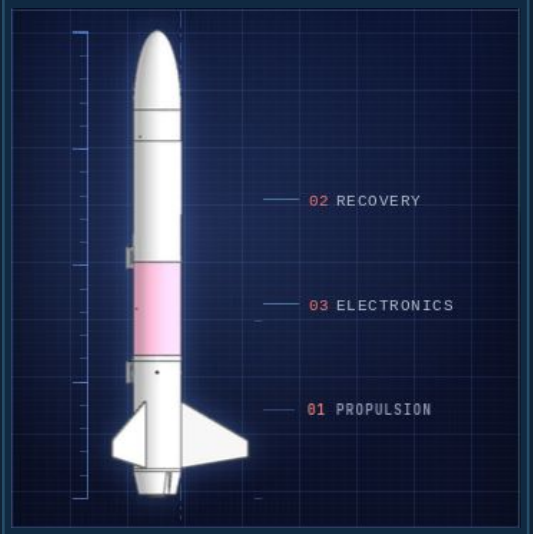
## NEMESIS




**Competition rocket**

EuRoC 2025

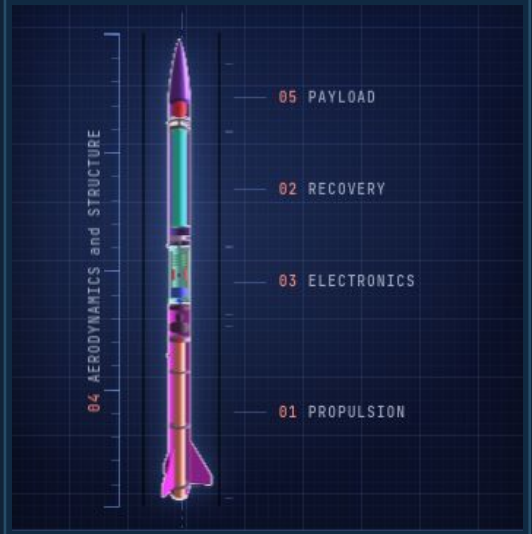
## FRED




**Flight Research Experimental Device**

Experimental platform

## ATLAS

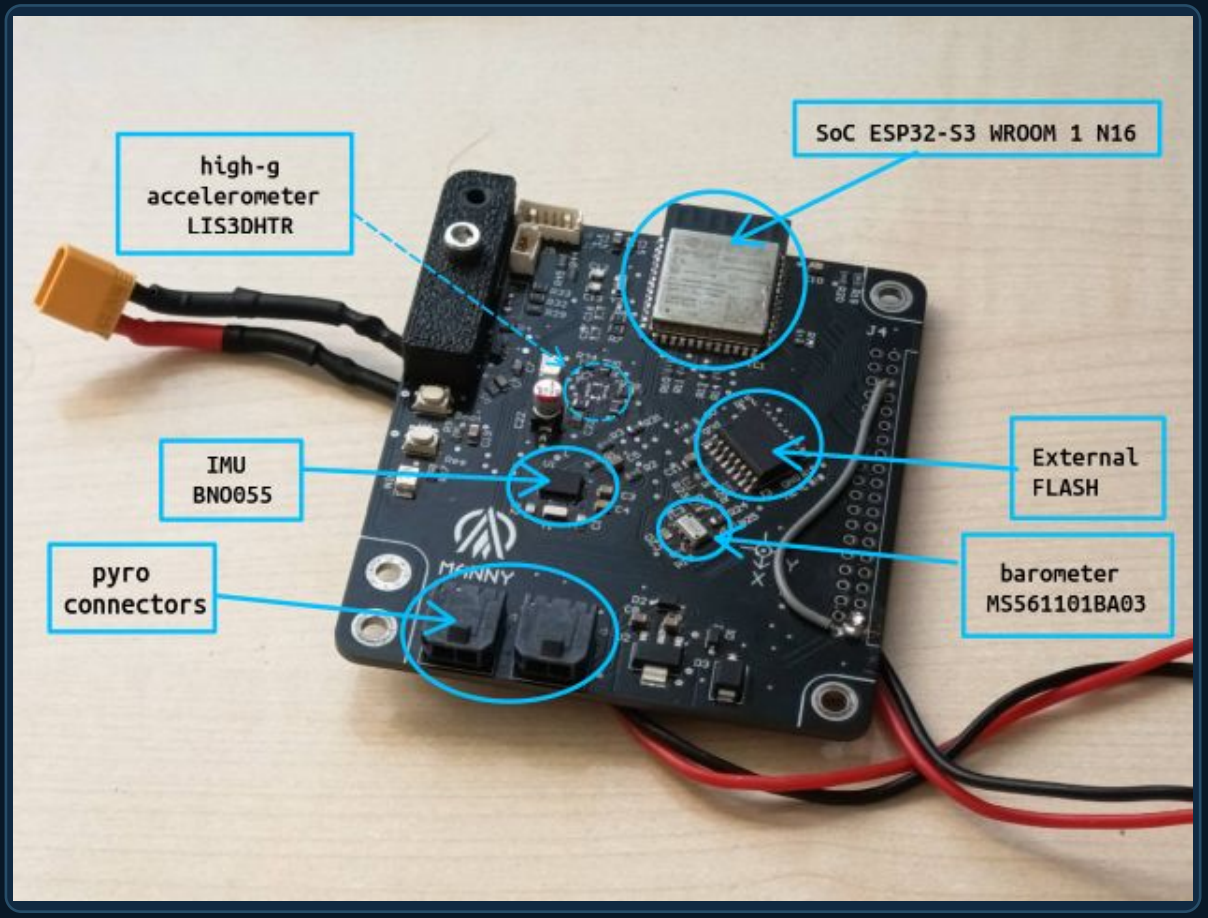



**Next-generation rocket**

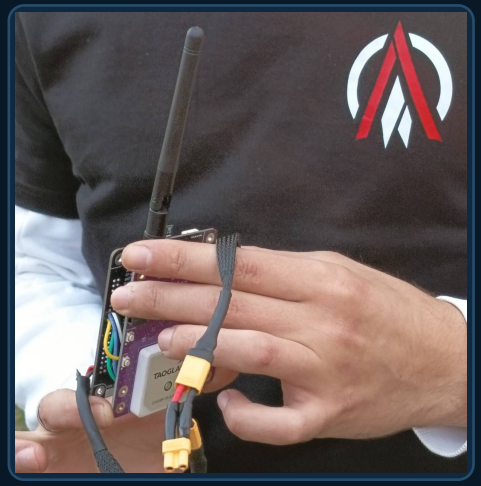
Under development

# Meet the Avionics

Manny runs Atlas; AMI is its SPI-connected antenna board.



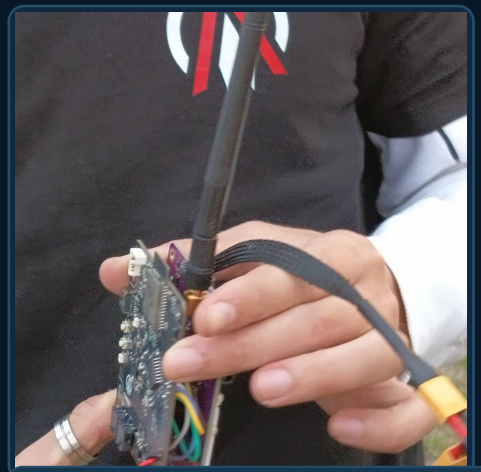

MANNY / ESP32-S3 FLIGHT COMPUTER



## MANNY + AMI

Connected over SPI

View 01



## ELECTRONICS BAY

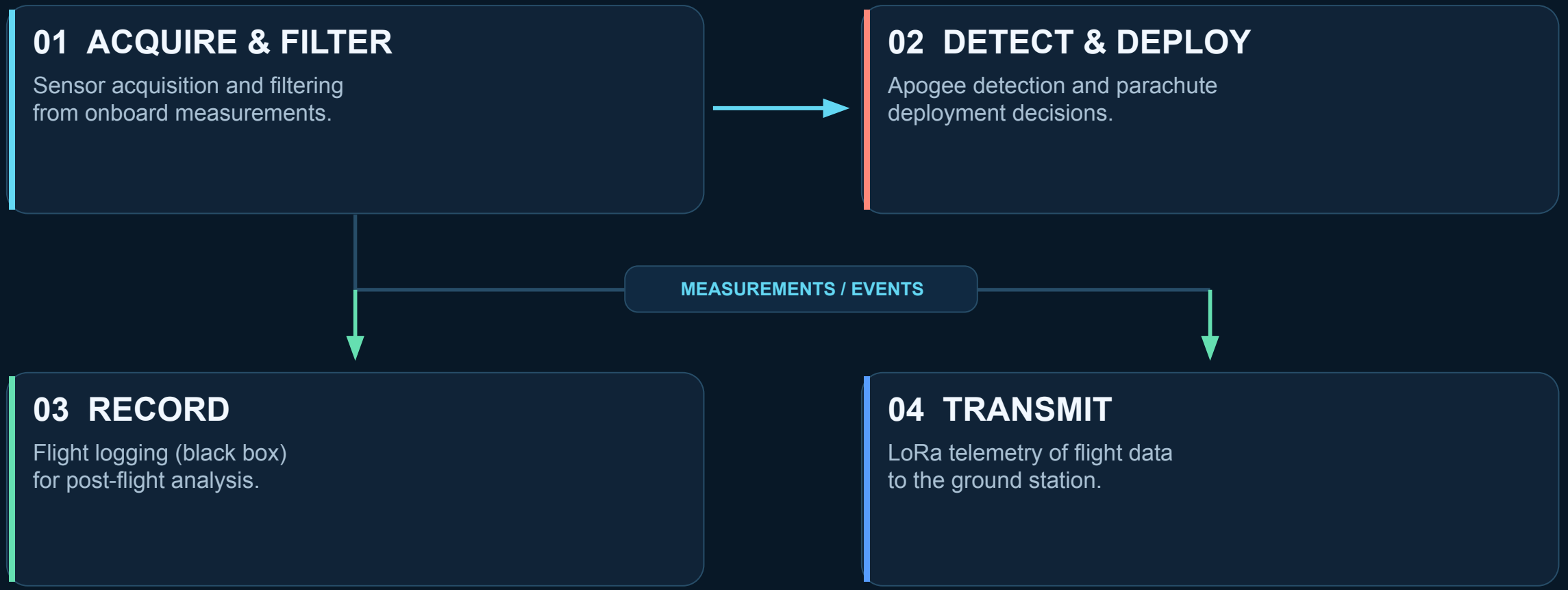
SPI-linked avionics assembly

View 02

ATLAS on MANNY · AMI over SPI

# What Does the Software Team Do?

Four responsibilities. One embedded flight system.




A high-level map of responsibilities — implementation details come later.

ESP-IDF BUILD DOCS ↗

# Exploring an ESP-IDF Project

Start at the repository root — everything else is a zoom from here.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/

atlas/
├── build
├── CMakeLists.txt
├── components
├── convert_sysincludes.py
├── data_analysis
├── dependencies.lock
├── docs
├── Doxyfile
├── flash_logs
├── include
├── LICENSE
├── main
├── managed_components
├── partitions.csv
├── README.md
├── sdkconfig
├── sdkconfig.defaults
├── sdkconfig.old
├── test
└── trace.txt
```

*User-provided local tree snapshot*

**ONE PROJECT · MANY ROLES**

**BUILD RULES** — **CMakeLists.txt · dependencies.lock**

Set the build graph and dependencies

**APPLICATION** — **main/ · include/**

Startup path and common headers

**SUBSYSTEMS** — **components/ · managed_components/**

Our modules and external dependencies

**DEVICE CONFIG** — **sdkconfig · partitions.csv**

Compile-time options and flash map

*build/ contains generated output; not every visible file is source code.*

ESP-IDF BUILD DOCS ↗

# How Does ESP-IDF Build a Project?

ESP-IDF v5.5.6 · idf.py is the everyday command-line entry point.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/

atlas/
├── build
├── CMakeLists.txt
├── components
├── convert_sysincludes.py
├── data_analysis
├── dependencies.lock
├── docs
├── Doxyfile
├── flash_logs
├── include
├── LICENSE
├── main
├── managed_components
├── partitions.csv
├── README.md
├── sdkconfig
├── sdkconfig.defaults
├── sdkconfig.old
├── test
└── trace.txt
```

**01** **idf.py build** — Developer command

↓

**02** **CMake** — Configure targets & components

↓

**03** **Compile** — Build component libraries

↓

**04** **Link** — Produce firmware ELF / BIN

**idf.py flash → transfer the built image to the ESP32-S3**

# Firmware Configuration

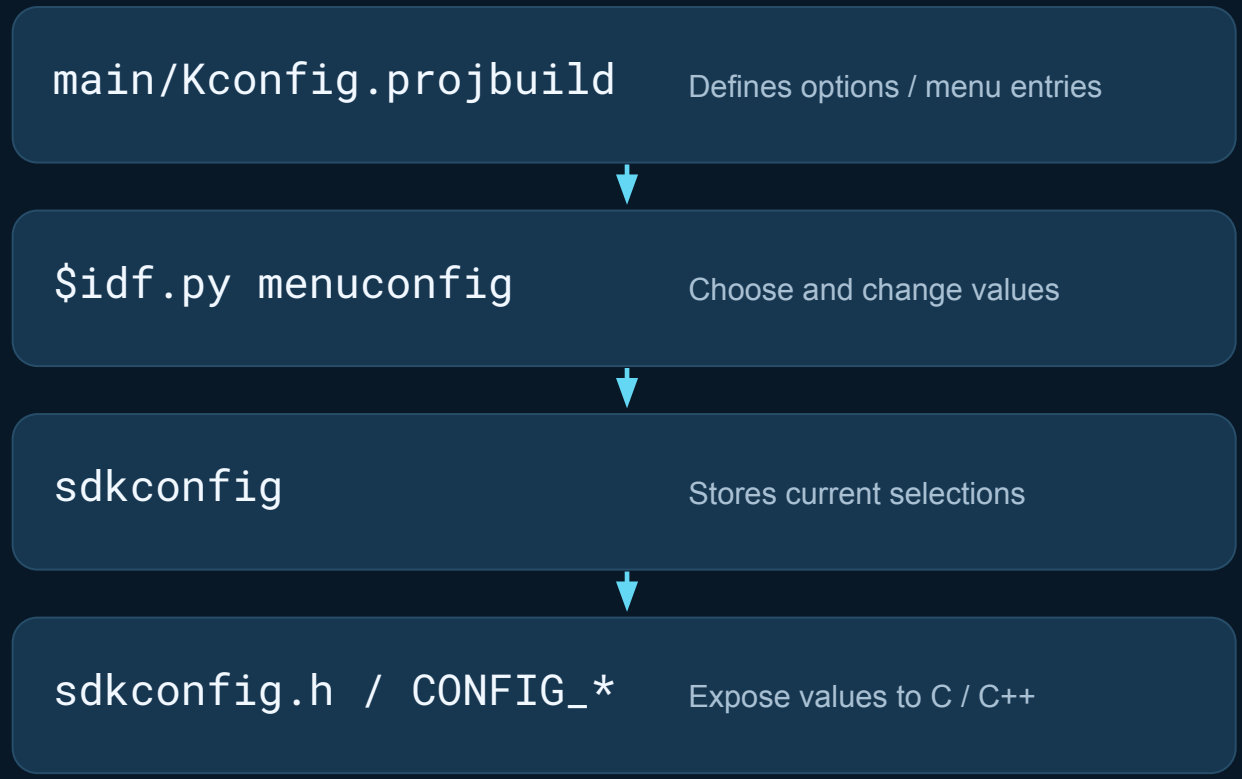
Kconfig defines the choices. sdkconfig records the selected build settings.

## TERMINAL / REPOSITORY

```
$ tree -L 1 atlas/

atlas/
├── build
├── CMakeLists.txt
├── components
├── convert_sysincludes.py
├── data_analysis
├── dependencies.lock
├── docs
├── Doxyfile
├── flash_logs
├── include
├── LICENSE
├── main
├── managed_components
├── partitions.csv
├── README.md
├── sdkconfig
├── sdkconfig.defaults
├── sdkconfig.old
├── test
└── trace.txt
```

## CONFIGURATION FLOW




sdkconfig.defaults → baseline values when creating configuration

Example: COMPILE_BOARD_MANNY, TELEMETRY_LINK_ANTENNA_BOARD

ESP-IDF BUILD DOCS ↗

# The Flash Partition Table

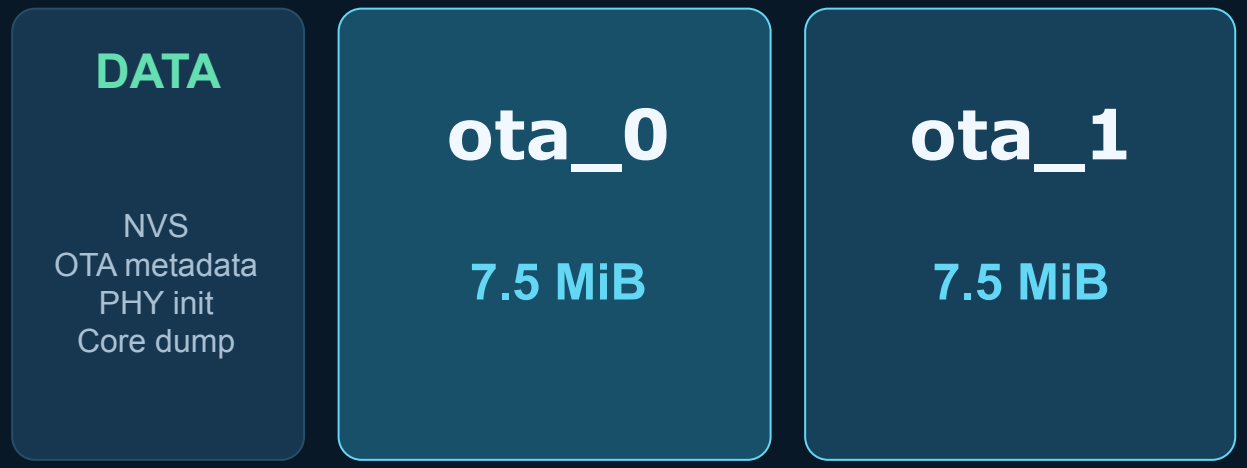
partitions.csv defines how the ESP32-S3's 16 MB internal flash is allocated.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/

atlas/
├── build
├── CMakeLists.txt
├── components
├── convert_sysincludes.py
├── data_analysis
├── dependencies.lock
├── docs
├── Doxyfile
├── flash_logs
├── include
├── LICENSE
├── main
├── managed_components
├── partitions.csv
├── README.md
├── sdkconfig
├── sdkconfig.defaults
├── sdkconfig.old
├── test
└── trace.txt
```

**INTERNAL FLASH / 16 MB**




*Schematic, not to scale · Bootloader and partition-table regions also exist.*

## WHY TWO APP SLOTS?

One can run while the other receives an OTA firmware update.

**External flight-log flash is NOT one of these ESP-IDF partitions.**

# main/ vs components/

Source organization is not the same thing as runtime scheduling.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/

atlas/
├── build
├── CMakeLists.txt
├── components
├── convert_sysincludes.py
├── data_analysis
├── dependencies.lock
├── docs
├── Doxyfile
├── flash_logs
├── include
├── LICENSE
├── main
├── managed_components
├── partitions.csv
├── README.md
├── sdkconfig
├── sdkconfig.defaults
├── sdkconfig.old
├── test
└── trace.txt
```

## main/

Application entry point, startup, object wiring and mode selection.

## components/

Submodules, services, device drivers, state logic and implementations.

**CMake connects both at build time**

Next: open main/ and follow execution from the first function.

# Where Does Execution Begin?

The first source file to open is main/app_entry.cpp.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/main/

atlas/main/
├── app_entry.cpp
├── CMakeLists.txt
├── dashboard
├── hil
├── idf_component.yml
├── Kconfig.projbuild
├── main.cpp
├── main_hil.cpp
└── PlotPosition.py
```

**ENTRY POINT / main/app_entry.cpp**

**ESP-IDF startup** — Framework initializes the application

↓

`app_main()` — Defined in app_entry.cpp

↓

`initArduino()` — Initialize the Arduino layer

↓

`setupSelectedMode()` — Select Flight or HIL

↓

`loopSelectedMode()` — Run the selected loop continuously

# Flight Mode vs HIL Mode

One app_main(); a compile-time choice selects which startup path is built.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/main/

atlas/main/
├── app_entry.cpp
├── CMakeLists.txt
├── dashboard
├── hil
├── idf_component.yml
├── Kconfig.projbuild
├── main.cpp
├── main_hil.cpp
└── PlotPosition.py
```

**SELECTION IN ROOT CMakeLists.txt**

**AURORA_HIL_ENABLED = OFF / ON**

**FLIGHT**

main.cpp

setupFlight() / loopFlight()

**HIL**

main_hil.cpp

setupHil() / loopHil()

**With HIL OFF, main_hil.cpp is not compiled.**

# Starting RocketFSM

Move from application setup to an independently scheduled FSM task.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/main/

atlas/main/
├── app_entry.cpp
├── CMakeLists.txt
├── dashboard
├── hil
├── idf_component.yml
├── Kconfig.projbuild
├── main.cpp
├── main_hil.cpp
└── PlotPosition.py
```

**NORMAL FLIGHT STARTUP**

| | |
|---|---|
| `board.init()` | Hardware, status & configuration |
| **Components** | Sensors, logger & RocketModel |
| `RocketFSM(...)` | Construct the state-machine object |
| `rocketFSM->init()` | Prepare managers, states and transitions |
| **Wait for arming** | Precondition before starting the FSM |
| `rocketFSM->start()` | Create FSM_Task (FreeRTOS) |

**init() configures; start() launches execution.**

# What Is a Finite State Machine?

A finite state machine models behavior as states connected by permitted transitions.

## REPOSITORY / CONTROL (SELECTED PATHS)

```
atlas/components/control/

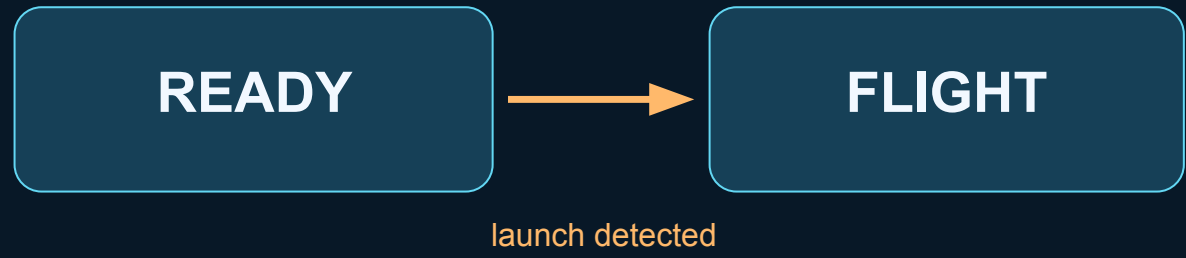
atlas/components/control/
├── CMakeLists.txt
└── src/
    ├── RocketFSM.cpp
    ├── RocketFSM.hpp
    ├── states/
    │   ├── StateAction.hpp
    │   └── TransitionManager.cpp
    └── tasks/
        └── TaskManager.cpp
```

*Selected paths, not the full tree output*

## THREE IDEAS

| STATE | TRANSITION | ACTION |
|---|---|---|
| Current operating mode | Condition or event | Work triggered by a state |

## SMALL EXAMPLE / CONCEPTUAL




**Real implementation: RocketFSM + TransitionManager + state actions**

# Atlas Flight States

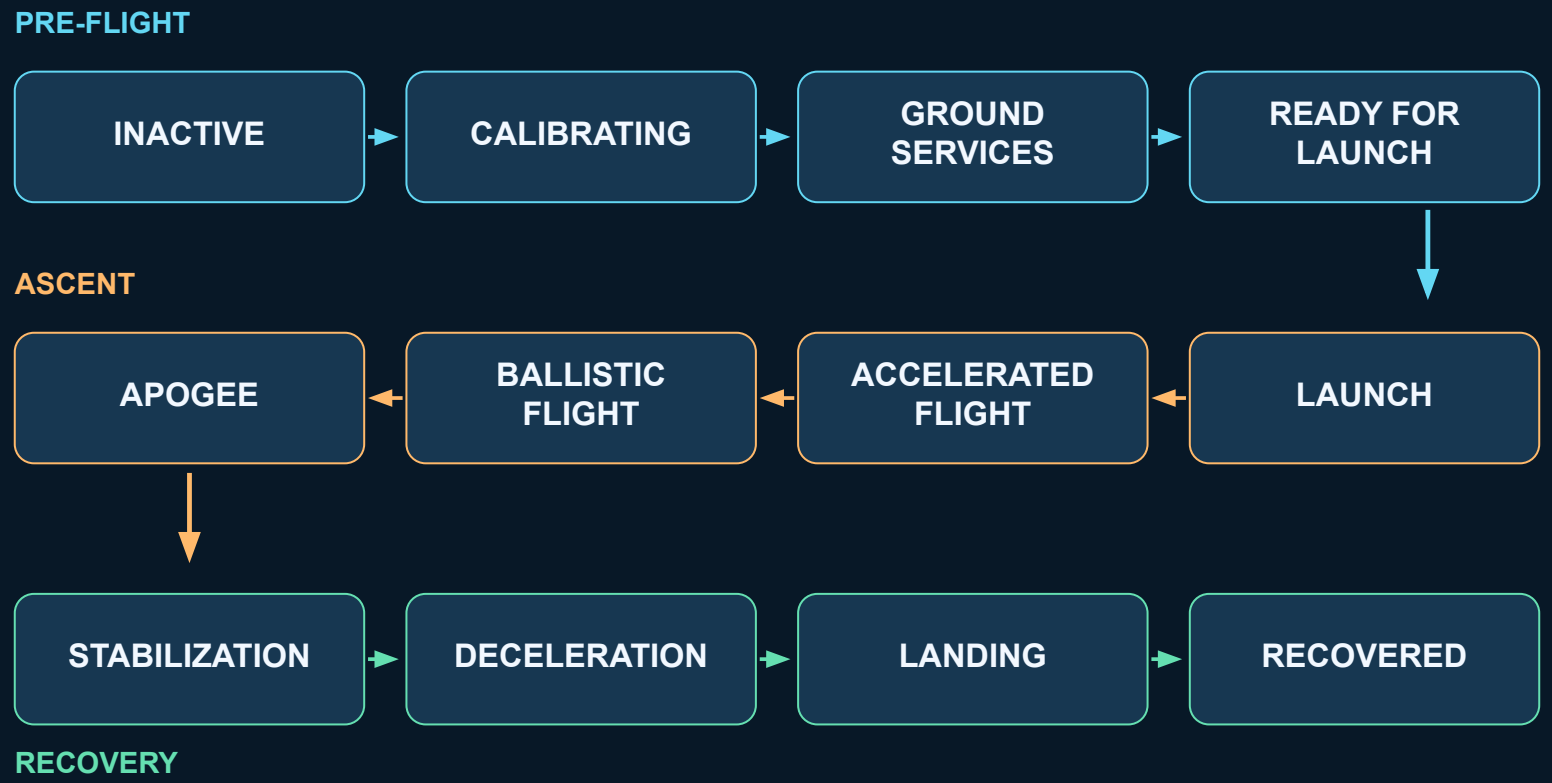
Nominal state sequence, grouped for readability. Emergency recovery is a separate path.

**REPOSITORY / CONTROL (EXCERPT)**

atlas/components/control/

```
atlas/components/control/
├── CMakeLists.txt
└── src/
    ├── RocketFSM.cpp
    ├── RocketFSM.hpp
    ├── states/
    │   ├── StateAction.hpp
    │   └── TransitionManager.cpp
    └── tasks/
        └── TaskManager.cpp
```

*FSM transition definitions*




**NON-NOMINAL: EMERGENCY_RECOVERY on reboot with flight in progress**

# Exploring components/

A codebase organized by responsibility — not a flat list of C++ files.

TERMINAL / REPOSITORY

```
$ tree -L 1 atlas/components/

atlas/components/
├── common/
├── control/
├── global/
├── logger/
├── model/
├── persistence/
├── protocols/
├── README
├── sensors/
├── status/
├── telemetry/
├── tests/
└── third_party/
```

*Real directories · examples highlighted*

A CRUCIAL DISTINCTION

## Group folders

`logger/` `sensors/` `telemetry/`

Organize related components; not automatically build components themselves.

## Actual ESP-IDF components

`status/` `logger/rocket_logger/`

Recognized by a component-level CMakeLists.txt

**Atlas detail**

**The root CMakeLists.txt uses EXTRA_COMPONENT_DIRS**

to register nested component directories explicitly.

# Anatomy of an ESP-IDF Component

The status module is a concrete example from Atlas.

**TERMINAL / REPOSITORY**

```
$ tree -L 2 atlas/components/status/

atlas/components/status/
├── CMakeLists.txt
├── library.json
└── src/
    ├── BuzzerController.cpp
    ├── BuzzerController.hpp
    ├── LEDController.cpp
    ├── LEDController.hpp
    ├── StatusManager.cpp
    └── StatusManager.hpp
```

*All src files kept together by convention*

**CMakeLists.txt**

**REQUIRED BY ESP-IDF**

Register source files, include paths and dependencies.

**library.json**

**PLATFORMIO METADATA**

Useful manifest, but not an ESP-IDF requirement.

**src/**

**OPTIONAL DIRECTORY**

Our convention: .hpp and .cpp live side by side.

**REAL BUILD REGISTRATION**

```
file(GLOB SRCS "src/*.cpp" "src/*.c")
idf_component_register(SRCS ${SRCS}
  INCLUDE_DIRS "src"  REQUIRES global data)
```

# Understanding managed_components/

Dependencies downloaded by IDF Component Manager — not code you edit in place.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/

atlas/
├── build
├── CMakeLists.txt
├── components
├── convert_sysincludes.py
├── data_analysis
├── dependencies.lock
├── docs
├── Doxyfile
├── flash_logs
├── include
├── LICENSE
├── main
├── managed_components
├── partitions.csv
├── README.md
├── sdkconfig
├── sdkconfig.defaults
├── sdkconfig.old
├── test
└── trace.txt
```

*managed_components/ deliberately not expanded*

**MANAGED DEPENDENCY WORKFLOW**

`main/idf_component.yml` — Declares external dependencies

↓

**IDF Component Manager** — Resolves versions and downloads code

↓

`managed_components/` — Build-time dependencies placed here

`dependencies.lock` — Pins resolved versions

**DIFFERENT:** `components/third_party/` **= vendored libraries in the repo**

# Component Responsibilities · Core

What each folder owns, at a glance.

TERMINAL / REPOSITORY

```
$ tree -L 1 atlas/components/

atlas/components/
├── common/
├── control/
├── global/
├── logger/
├── model/
├── persistence/
├── protocols/
├── README
├── sensors/
├── status/
├── telemetry/
├── tests/
└── third_party/
```

*Same directory tree · different focus*

| | |
|---|---|
| common/data | Shared measurement data types (IMU, pressure, GPS, etc.). |
| control | RocketFSM, FreeRTOS task classes and their lifecycle. |
| sensors | Device-specific drivers and sensor abstractions. |
| model | RocketModel: shared data, synchronization and resource access. |
| logger | RocketLogger queues + log serialization. |
| persistence | Storage APIs and Flash / SD backends. |

*Important: a directory name is not itself an execution context.*

# Component Responsibilities · Support

The remaining subsystems complete the flight software.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 atlas/components/

atlas/components/
├── common/
├── control/
├── global/
├── logger/
├── model/
├── persistence/
├── protocols/
├── README
├── sensors/
├── status/
├── telemetry/
├── tests/
└── third_party/
```

*Same directory tree · different focus*

| | |
|---|---|
| `global` | Configuration, pins, shared flags and serial diagnostics. |
| `protocols` | Reusable I²C and SPI bus communication wrappers. |
| `telemetry` | TelemetryLink.hpp: AMI (SPI) or E220 (UART). |
| `status` | StatusManager, LED and buzzer indication. |
| `tests` | Hardware/functional test routines used during development. |
| `third_party` | Vendored external libraries included in the repository. |

*SPI driver: telemetry/radio_lora/src/AntennaBoardTransmitter.cpp*

# C++ Objects vs FreeRTOS Tasks

A task class implements behavior; the RTOS provides an execution context.

**TERMINAL / REPOSITORY**

```
$ tree -L 1 .../tasks/  (excerpt)

atlas/components/control/src/tasks/
├── BaseTask.cpp
├── BaseTask.hpp
├── StorageLoggingTask.cpp
├── StorageLoggingTask.hpp
├── TaskManager.cpp
├── TaskManager.hpp
└── ITask.hpp
```

*Selected task files from a larger directory*

**1 OBJECT CREATION**

`TaskManager::initializeTasks()`

creates a StorageLoggingTask instance

**2 SCHEDULER HANDOFF**

`BaseTask::start(config)`

calls xTaskCreate() / xTaskCreatePinnedToCore()

**3 EXECUTION IN THE FREERTOS TASK**

`taskWrapper()` → `internalTaskFunction()`

→ `virtual taskFunction()` `while (running) { ... }`

**The object persists; the RTOS task starts, runs and eventually exits.**

# From Sensor Reading to Black Box

Follow one record across module boundaries — each layer has a different job.

**REPOSITORY / SELECTED PATHS**

```
atlas/components/ (curated)

atlas/components/  [selected paths]
├── control/src/tasks/
│   ├── SensorTask.cpp
│   └── StorageLoggingTask.cpp
├── logger/rocket_logger/src/
│   └── RocketLogger.cpp
├── model/
│   └── RocketModel.cpp
└── persistence/
    ├── storage/IStorage.hpp
    ├── storage/MirrorStorage.cpp
    ├── flash/Flash.cpp
    └── SD/SD-master.cpp
```

*Selected paths, not a literal tree command output*

| | |
|---|---|
| `SensorTask` | Reads via `RocketModel`; enqueues a sample |
| ↓ | |
| `RocketLogger` | Buffers records in a FreeRTOS queue |
| ↓ | |
| `StorageLoggingTask` | Consumes batches inside `taskFunction()` |
| ↓ | |
| `RocketModel` | `storageAppendFile()` guards storage with mutex |
| ↓ | |
| `IStorage / MirrorStorage` | Common file API; forwards writes to backends |
| ↓ | |
| `Flash / SD` | Physical black-box storage implementations |

**Storage writes are mirrored sequentially, not by parallel tasks.**

# TaskManager: Who Runs What?

The FSM chooses the required task set; TaskManager owns and starts the task objects.

**TERMINAL / REPOSITORY**

```
$ tree .../control/src/ (excerpt)

atlas/components/control/src/
├── RocketFSM.cpp
├── states/
│   └── StateAction.hpp
└── tasks/
    ├── TaskConfig.hpp
    ├── TaskManager.hpp
    ├── TaskManager.cpp
    ├── BaseTask.cpp
    └── SensorTask.cpp
```

*Selected files, same navigation model*

### `TaskManager::initializeTasks()`

Creates task objects once; hardware-dependent tasks may be skipped.

### `RocketFSM::setupStateActions()`

Registers a TaskConfig list for each RocketState.

### `RocketFSM::transitionTo()`

Compares old vs new TaskType sets and calls startTask()/stopTask().

**TaskConfig contains type, name, stack, priority and core.**

*TaskManager manages task objects; it is not itself every task's thread.*

# Task Activation by Flight Phase

FSM task assignments, as configured in setupStateActions() — not a runtime health report.

**TERMINAL / REPOSITORY**

```
$ tree .../control/ (excerpt)

atlas/components/control/src/
├── RocketFSM.cpp
├── states/StateAction.hpp
└── tasks/
    ├── TaskConfig.hpp
    └── TaskManager.cpp
```

*Source: setupStateActions()*

**Configured task types · flight build**

| | CAL | GROUND | READY | ASCENT | DESCENT | DONE | EMERG |
|---|---|---|---|---|---|---|---|
| Sensor | ● | ● | ● | ● | ● | ● | ● |
| GPS* | ● | ● | ● | ● | ● | ● | — |
| Altitude | — | ● | ● | ● | ● | — | — |
| Storage | — | ● | ● | ● | ● | — | — |
| Telemetry | ● | ● | ● | ● | ● | ● | — |
| Ground Svc. | — | ● | — | — | — | — | — |
| Airbrakes | — | — | — | — | — | — | — |

## GPS TASK IS LEGACY

**AMI now manages GPS; GPS data is included in telemetry sent over LoRa.**

GpsTask remains in the FSM configuration but is obsolete on the current Manny + AMI system.

● Assigned in FSM — Not assigned | Normal flight build; HIL replaces acquisition tasks.

# Task Diff: A Transition Without Restarts

The FSM compares task types, so tasks shared by both states keep running.

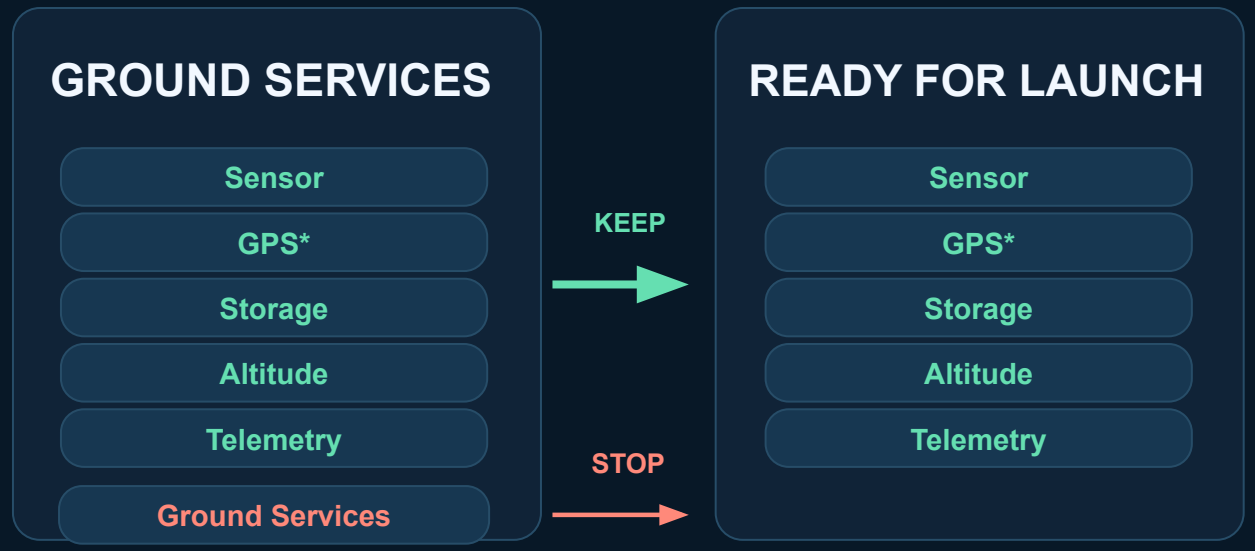
**TERMINAL / REPOSITORY**

```
$ tree .../control/ (excerpt)

atlas/components/control/src/
├── RocketFSM.cpp
└── tasks/
    ├── TaskManager.cpp
    └── TaskConfig.hpp
```

*transitionTo() compares TaskType, not task name*

**EXAMPLE: GROUND_SERVICES → READY_FOR_LAUNCH**




**Task names may change across states; existing TaskType instances are kept.**

* GpsTask is legacy: AMI now owns GPS and includes it in LoRa telemetry.

*READY also locks configuration and activates the prepared flight log — separate from Task Diff.*

# One Interface, Different Flight Computers

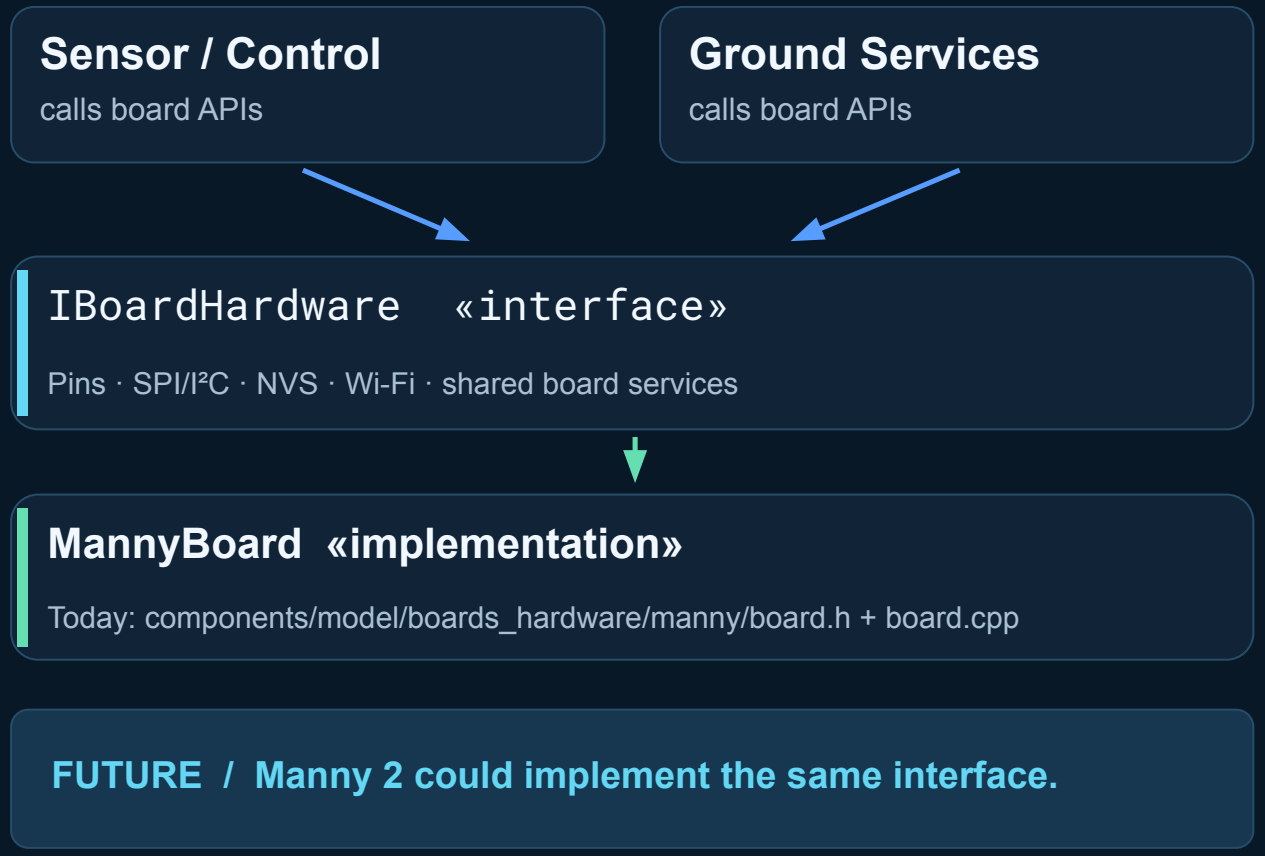
IBoardHardware keeps board-specific details behind a common API.

**TERMINAL / REPOSITORY**

```
$ tree .../model/ (excerpt)

atlas/components/model/
├── RocketModel.cpp
├── boards_hardware/
│   ├── IBoardHardware.hpp
│   └── manny/
│       ├── board.h
│       └── board.cpp
└── RocketModel.hpp
```

*Actual names: board.h + board.cpp*




*Design goal, not a guarantee: existing Manny-specific constants still need a migration audit.*

# Trace a Sensor Sample: Acquisition to Queue

Sensor drivers publish measurements; the logger receives copies for recording.

**REPOSITORY / SELECTED PATHS**

```
atlas/components/ (curated)

atlas/components/  [selected paths]
├── sensors/BNO055/src/
├── common/data/src/
├── model/RocketModel.cpp
├── control/src/tasks/
│   └── SensorTask.cpp
└── logger/rocket_logger/src/
    └── RocketLogger.cpp
```

*A sample crosses folders — follow the APIs*

**`SensorTask::taskFunction()`**

calls `RocketModel::updateBNO055()`

**`RocketModel`**

calls sensor driver, publishes a locked snapshot

**`SensorTask`**

reads snapshot with `getBNO055Data()`

**`RocketLogger::logSensorData()`**

enqueues a `LogPayload` into `_logQueue`

**50 Hz acquisition → sensor records typically queued at 25 Hz**

*Shared state (RocketModel) and recording buffer (RocketLogger) are different responsibilities.*

# Trace a Sensor Sample: JSONL to Flash

A serialization function is selected at startup; the storage task drains and writes batches.

**REPOSITORY / SELECTED PATHS**

```
atlas/ (curated)

atlas/ [selected paths]
├── main/main.cpp
└── components/
    ├── logger/rocket_logger/src/
    |   └── RocketLogger.cpp
    ├── logger/serializations/src/
    |   └── PayloadSerializer.hpp
    ├── control/src/tasks/
    |   └── StorageLoggingTask.cpp
    ├── model/RocketModel.cpp
    └── persistence/
        ├── storage/
        └── flash/
```

*Each arrow crosses an API boundary*

```
logger->setSerializer(PayloadSerializers::toJson);
```

**RocketLogger::consumeBatch()**

drains queues, invokes the chosen serializer

↓

**PayloadSerializers::toJson()**

writes one serialized JSON object per record

↓

**StorageLoggingTask::taskFunction()**

writes the filled buffer through RocketModel

↓

**IStorage → MirrorStorage → Flash / SD**

portable storage API; backends written sequentially

**JSONL = one JSON object per line · chosen in main.cpp**

# Real Flight Logs

Recorded measurements turn into interpretable flight traces.

## REPOSITORY / ANALYSIS

```
$ tree -L 1 atlas/data_analysis/

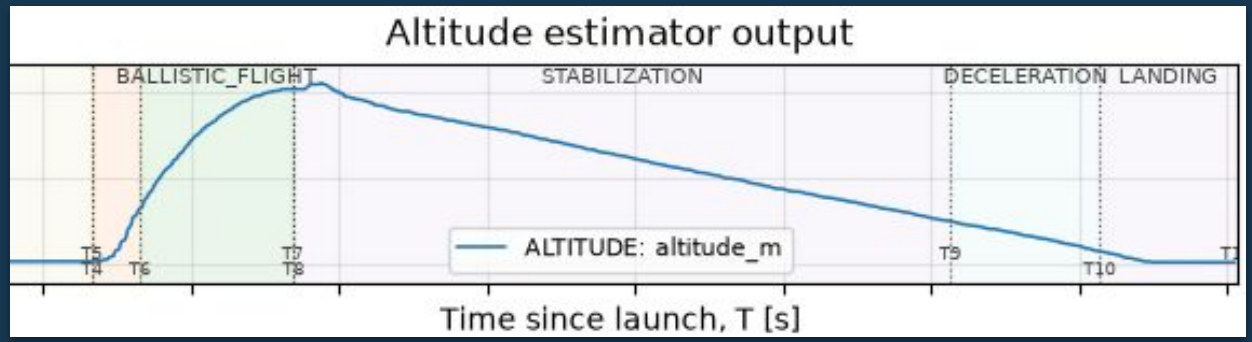
atlas/data_analysis/
├── analyze_flight_log.py
├── AnalyzeJSONs.ipynb
└── extract_flash_logs.py
```

## FROM THE BLACK BOX

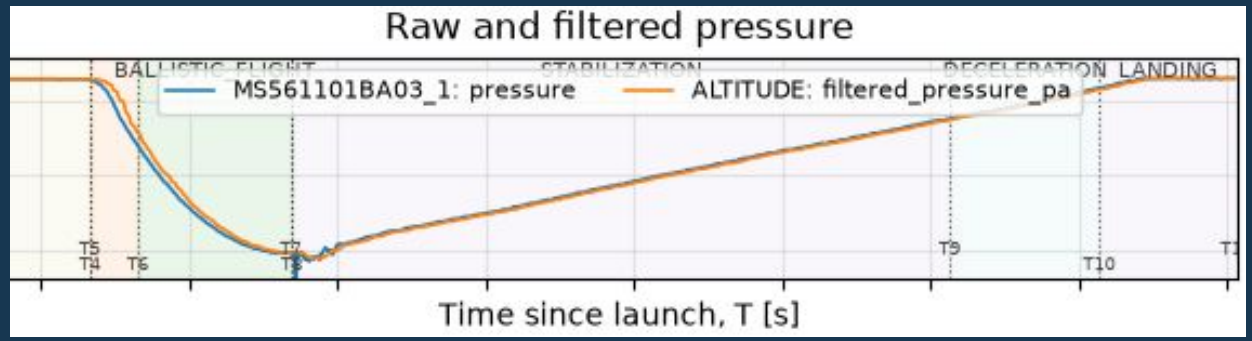
JSONL flight records → offline Python analysis → plots

Look for phase transitions and sensor trends, not absolute altitude.

## ALTITUDE ESTIMATOR OUTPUT




## RAW AND FILTERED PRESSURE




Y-axis values intentionally omitted • Focus: trajectory shape, noise and flight phases

# Ground Services: Preflight Observatory

A dedicated FSM state and FreeRTOS task provide a browser-based operator interface.

**TERMINAL / REPOSITORY**

```
$ tree .../control/src/ (excerpt)

atlas/components/control/src/
├── RocketFSM.cpp
└── tasks/
    ├── GroundServicesTask.cpp
    ├── GroundServicesTask.hpp
    ├── RuntimeConfig.cpp
    └── web/
```

*HTTP server and UI live inside GroundServicesTask*

**GROUND_SERVICES** — FSM state

↓

**GroundServicesTask** — FreeRTOS task

↓

**ESP-IDF HTTPD + SoftAP** — Browser interface

**OPERATOR FEATURES**

| | | |
|---|---|---|
| INFO / HEALTH | LIVE DATA | CONFIG / NVS |
| OTA UPDATE | FILES / DOWNLOAD | TESTS / LOGS |

*Monitoring is read-only; critical actions require operator authentication.*

# Ground Services — Live Data

A real Manny board and the onboard web interface, side by side.

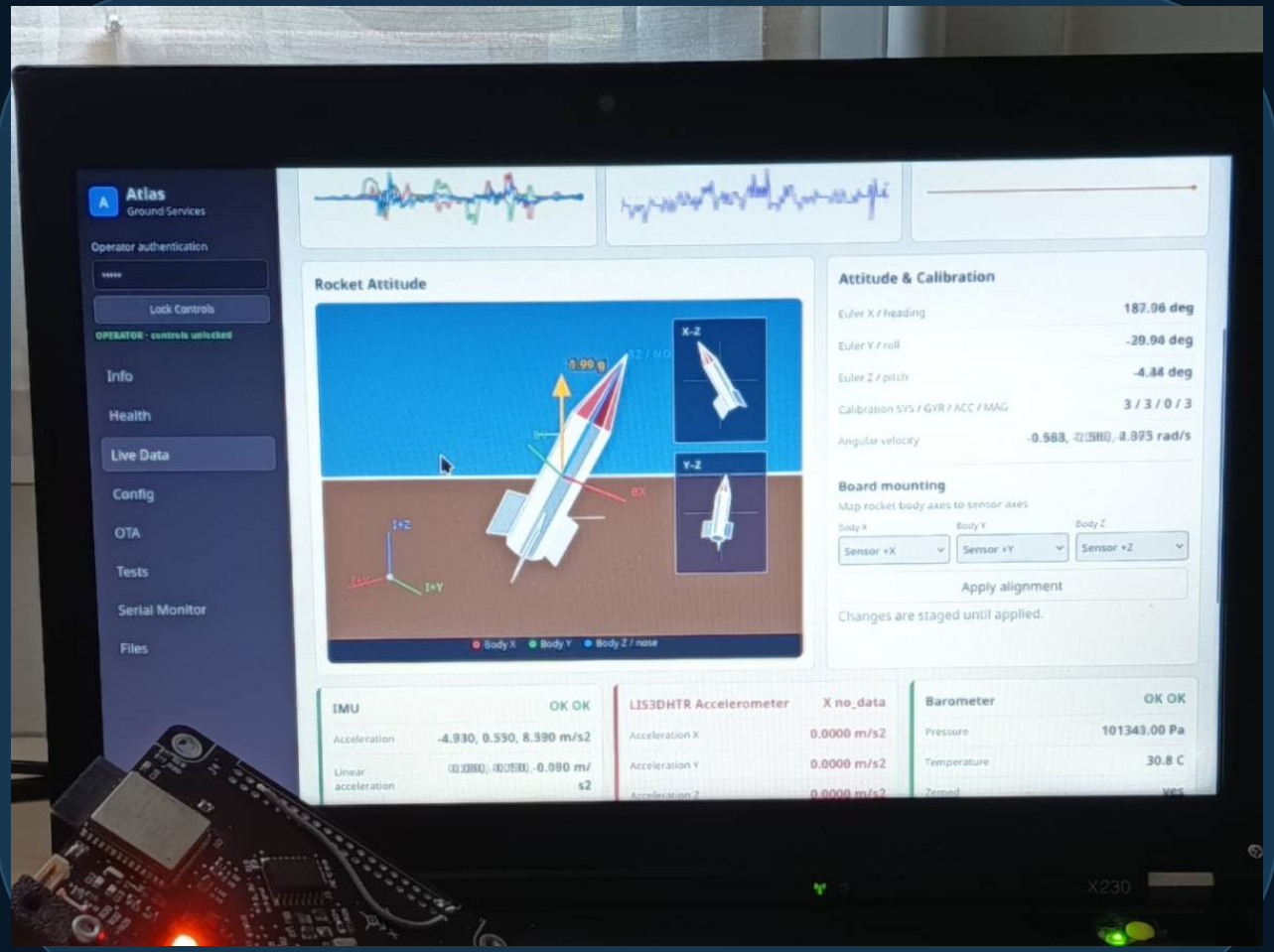
## REPOSITORY / CONTROL

```
$ tree .../control/src/tasks/

atlas/components/control/src/
└── tasks/
    ├── GroundServicesTask.cpp
    ├── GroundServicesTask.hpp
    ├── RuntimeConfig.cpp
    └── web/
```

## PRE-FLIGHT OBSERVABILITY

Health • live sensors • rocket orientation • calibration




LIVE DATA / BOARD ORIENTATION / SENSOR HEALTH

# RuntimeConfig: Edit, Persist, Lock

Mission parameters are configured on the ground and frozen before flight.

## TERMINAL / REPOSITORY

```
$ tree .../tasks/ (excerpt)

atlas/components/control/src/tasks/
├── GroundServicesTask.cpp
├── RuntimeConfig.cpp
├── RuntimeConfig.hpp
└── web/
```

*Actual struct: RuntimeConfig.hpp*

## STRUCT DOMAINS

- mission
- recovery
- flight
- telemetry
- airbrakes
- storage_logging
- altitude
- actuators
- calibration
- flight_transitions

metadata: schema_version · config_revision · config_locked · checksum

## LIFECYCLE

**EDIT** — Ground UI → **VALIDATE** — check ranges → **PERSIST** — NVS blob → **LOCK** — READY flight

*Only approved fields are editable; calibration/telemetry can be inspection-only. Flight tasks read a locked snapshot.*

# HIL: The Same FSM with Simulated Sensors

Firmware logic runs on Manny while RocketPy supplies measurements over TCP.

**TWO ENDS OF THE HIL LOOP**

```
atlas/ (curated)

atlas/  [selected paths]
├── main/
|   └── hil/
|       ├── README.md
|       ├── hil_rocketpy.py
|       ├── hil_communication.py
|       └── config/
└── components/control/src/tasks/
    └── hil/
        └── HilSimulationTask.cpp
```

*main/hil/README.md is the operational guide*

**RocketPy simulator (Python)**

`main/hil/hil_rocketpy.py` · 50 Hz sampling

↓

**TCP :5000 → HilSimulationTask**

`components/control/src/tasks/hil/` · replaces SensorTask/GpsTask

↓

**RocketModel → RocketFSM**

Injected sensor snapshots drive the same state transitions.

↶ **FC events / actuator commands are returned to the simulator.**

**Build switch: AURORA_HIL_ENABLED = ON / OFF in root CMakeLists.txt**

*Reference: main/hil/README.md · HIL is excluded from normal flight builds.*

# Development Environment — Getting Started

Install the tools together, pin ESP-IDF v5.5.6, and verify your first Atlas build.

## SET UP YOUR WORKSTATION

**01 Install Visual Studio Code**

Desktop editor (Windows / Linux / macOS)

OFFICIAL DOWNLOAD ↗

**02 Add the ESP-IDF extension**

VS Code → Extensions → "ESP-IDF" (Espressif)

EXTENSION PAGE ↗

**03 Run ESP-IDF Installation Manager**

F1 → ESP-IDF: Open ESP-IDF Installation Manager

INSTALLATION GUIDE ↗

**04 Select ESP-IDF v5.5.6**

Install v5.5.6 and select it as the current setup

EIM DOCUMENTATION ↗

## VERIFY THE INSTALLATION

**ESP-IDF terminal / project root**

```
$ git clone https://github.com/AuroraRocketryTeam/atlas.git
$ cd atlas
$ idf.py --version
$ idf.py build
```

**CHECKPOINT**

**ESP-IDF version is v5.5.6 and the firmware builds.**

**Build on your PC; flashing/monitoring needs a Manny board.**

*For install problems, run ESP-IDF: Doctor Command in VS Code.*